

Assignment: Movie Recommendations with Collaborative Filtering

Unsupervised Learning

General Information

Work mode: Individual or groups (max 2 students)

Submission: Jupyter notebook (.ipynb) via email to diletta.abbonato@unito.it

Just send me the notebook :)

1 Introduction

You have a dataset of real movie ratings from real users. Your goal is to build a recommendation system using the techniques from the lectures: collaborative filtering and matrix factorisation.

This assignment is not about building the best possible recommender. It is about understanding the mechanics, interpreting the results, and connecting what you see to the concepts we covered in class (distances, similarity, dimensionality reduction, latent factors).

Important: Several tasks ask you to choose movies that *you* personally know and like. Your results will depend on these choices. In the notebook, explain *why* the recommendations make sense (or don't) based on your knowledge of the movies you picked.

2 Dataset

You will use the **MovieLens 25M** dataset:

<https://grouplens.org/datasets/movielens/25m/>

If your machine cannot handle 25M ratings, use the small version (100K ratings):

<https://grouplens.org/datasets/movielens/latest/> (ml-latest-small.zip)

The dataset contains explicit ratings from real users on real movies. You will need two files:

File	Columns
ratings.csv	userId, movieId, rating, timestamp
movies.csv	movieId, title, genres

3 Exploring the Data

Before building anything, understand what you're working with.

Tasks

1. Load both files and report: how many users, how many movies, how many ratings. Compute the **sparsity** of the user-item matrix (what percentage of cells are empty).
2. Plot the distribution of ratings. Is it symmetric? Are users generous or harsh on average?

3. Plot the distribution of **ratings per user** (how many movies does a typical user rate?) and **ratings per movie** (how many ratings does a typical movie get?). Use log scale if needed.
4. **Filter the data:** keep only users with at least 50 ratings and movies with at least 50 ratings. Report the new dimensions and sparsity. Explain why this filtering is necessary.

Questions to answer

Do you need to transform the ratings (e.g., log transformation, normalisation)? Why or why not? Think about what you learned regarding implicit vs explicit feedback.

4 Collaborative Filtering

Implement both user-based and item-based collaborative filtering. Use **cosine similarity** as the similarity metric.

Tasks

- 4.1 **Choose your anchor movies.** Pick **3 movies** that you personally know well and enjoy. They should be different from each other (e.g., don't pick three Marvel films). Write a sentence for each explaining why you chose it. These will be your anchor points throughout the assignment.
- 4.2 **Item-based CF.** Compute the item-item cosine similarity matrix. For each of your 3 anchor movies, find the 10 most similar movies. Present the results in a table.
- 4.3 **User-based CF.** Create a **fake user** profile: assign ratings to your 3 anchor movies (plus 5–10 other movies you know, to give the system enough signal). Find the 20 most similar real users to your fake profile. Generate 10 recommendations for your fake user.
- 4.4 **Compare.** Do the item-based and user-based approaches recommend the same movies? Show the overlap (or lack thereof).

Questions to answer

Look at the 10 most similar movies to each of your anchors. Do they make sense to you as a viewer? Are they similar in genre, mood, era, or something else? Is there a recommendation that surprises you? Try to explain why the algorithm thinks it's similar.

What happens if you change the number of neighbours k in user-based CF (try $k = 5, 20, 50, 100$)? How do the recommendations change? Why?

5 Matrix Factorisation (SVD)

Now compress the user-item matrix into latent factors using truncated SVD — the technique that won the Netflix Prize.

Tasks

- 5.1 Center the rating matrix by subtracting each user's mean rating. Explain why centering is necessary.
- 5.2 Apply truncated SVD with $k = 50$ latent factors. Reconstruct the predicted rating matrix. Generate 10 recommendations for your fake user profile.

- 5.3 Plot the singular values. How many factors capture most of the information?
- 5.4 In the latent space, find the 10 nearest neighbours to each of your 3 anchor movies (using the item factor vectors from SVD). Compare with the item-based CF results from Section 4.

Questions to answer

The SVD neighbours and the cosine-similarity neighbours may differ. When they disagree, which set of neighbours feels more “right” to you as a viewer? What might SVD be capturing that raw cosine similarity misses (or vice versa)?

What is the connection between SVD on the rating matrix and PCA? In what sense is SVD performing dimensionality reduction?

6 The Latent Space

SVD produces a latent representation for each movie. Let’s look at what this space has learned.

Tasks

- 6.1 Take the item latent vectors from SVD. Reduce them to 2D using UMAP (or t-SNE) and create a scatter plot coloured by the movie’s primary genre.
- 6.2 Locate your 3 anchor movies in the plot. Annotate them. What are their neighbours in the 2D space?
- 6.3 Pick a genre (e.g., Horror, Romance, Documentary). Are movies of this genre clustered together in the latent space, or are they scattered? Try at least two genres and compare.

Questions to answer

If genre clusters exist in the latent space, what does that tell you about user preferences? If they don’t exist (or only partially), why might that be?

Could you use this latent space to solve the cold-start problem for a new movie that has no ratings yet? How?

7 Practical Tips

- **Start with the small dataset** if you’re unsure about memory or compute. Everything works the same, just fewer users and movies.
- **Use sparse matrices** (`scipy.sparse.csr_matrix`) for the user-item matrix. With 25M ratings, a dense matrix may not fit in RAM.
- **Document your choices.** There is no single correct answer, but you must explain your reasoning.
- **Unexpected results are interesting.** If a method recommends something strange, don’t ignore it — investigate why.